Multi-Agent Systems Project

Design of a multi-agent system for automatic email responses using local LLMs

Master 2 Artificial Intelligence

Course : Multi-Agent Systems

General principle. The objective of the project is to design and implement in Java a multi-agent framework capable of assisting a user in automatically drafting replies to emails. The system must rely exclusively on language models running locally in order to limit the transmission of personal or confidential data to external services.

1. Context

Large language models (*Large Language Models*, or LLMs) can now automate many tasks related to natural language processing. Email drafting is, in particular, a natural use case for these technologies.

However, the use of remotely accessible LLM services raises an important issue : data protection.

An email may indeed contain personal, professional, administrative, scientific, or confidential information. In a university context, this may include, for example :

— exchanges between lecturers ;

— requests from students ;

— administrative information ;

— discussions related to research projects ;

— scientific documents that have not yet been published ;

— data relating to external partners.

In this project, we therefore choose to prioritize the use of **locally executed LLMs**.

This approach nevertheless presents a difficulty. Models that can run on a personal computer or local infrastructure often have more limited capabilities than very large models accessible through remote services.

We propose studying a different way of addressing this problem.

Rather than asking a single model to respond correctly to every situation, we aim to use the concepts of **Multi-Agent Systems (MAS)** to build an organization in which several agents have specialized roles and contexts.

The hypothesis examined in this project is therefore the following :

> *Can a multi-agent organization composed of specialized agents make it possible to use reasonably sized local LLMs effectively in order to produce responses adapted to different professional contexts ?*

2. Learning objectives

The objective of this project is to put into practice several concepts covered in the Multi-Agent Systems course.

By the end of the project, students should in particular be able to :

— identify the different roles that can be assigned to agents ;

— design a multi-agent organization ;

— define interactions between agents ;

— design task-routing and delegation mechanisms ;

— manage different knowledge and contexts depending on the agent ;

— integrate a local LLM into an application ;

— design a modular and extensible software architecture ;

— apply the fundamental principles of object-oriented programming ;

— use appropriate *Design Patterns* for the problem ;

— justify software-architecture and multi-agent design choices.

3. Problem to be solved

The goal is to develop an application capable of assisting a user in processing emails.

A single user may receive very different messages. In the case of a lecturer-researcher, for example, the following categories may be distinguished :

— emails from fellow lecturers ;

— emails from students ;

— emails from the administration ;

— emails related to a particular research project ;

— emails from collaborators or external partners.

A relevant response does not depend solely on the content of the most recently received message.

It may also depend on :

— the identity or role of the sender ;

— the subject of the email ;

— the history of the discussion thread ;

— previous exchanges with that person ;

— the associated professional context ;

— reference documents ;

— preferences defined by the user.

Your system must therefore be able to determine **which agent is best suited to process a given message**, and then provide that agent with the information required to generate a proposed response.

4. Proposed multi-agent organization

A minimal architecture may include two main categories of agents.

4.1. Routing agent

A first agent acts as a **router**, **coordinator**, or **dispatcher**.

When a new email is received, this agent must determine which specialized agent appears to be the most appropriate for producing the response.

This decision may in particular be based on :

— the sender's email address ;

— the known identity of the sender ;

— the sender's role ;

— the subject of the message ;

— the content of the message ;

— the discussion thread ;

— the various available metadata.

The classification must therefore **not necessarily rely solely on the sender's address**.
Indeed, the same person may contact the user in different contexts.
The routing agent then forwards the request and the necessary information to the selected specialized agent.

4.2. Specialized agents

The other agents specialize in certain categories of emails.
For example, the following agents may be considered :

— `StudentAgent` : responses to students ;

— `TeacherAgent` : responses to fellow lecturers ;

— `AdministrationAgent` : administrative exchanges ;

— `ResearchProjectAgent` : exchanges relating to a specific research project ;

— `GenericAgent` : handling situations that do not correspond to any particular category.

This list is purely illustrative.
You may propose a different organization if you believe it is more appropriate, provided that you can explain and justify it in the report.

5. Agent specialization

An agent's specialization must cover at least two dimensions.

5.1. The system prompt

Each agent must be able to have its own *system prompt*.
It may in particular define :

— its role ;

— the tone to use ;

— the information it must take into account ;

— the information it must never disclose ;

— how a response should be structured ;

— the user's specific preferences ;

— rules specific to the relevant context.

Ideally, the user should be able to modify these instructions without modifying the program's source code.

5.2. The context

Each specialized agent must also be able to have a **context of its own**.
This context may consist, for example, of :

— text files ;

— project-related documents ;

— information manually defined by the user ;

— old emails ;

— the history of a discussion thread ;

— other information you consider relevant.

It is therefore not expected that all agents have access to the same information.

6. Example : research project

Consider a user participating in a research project called `SuperMASProject`.
The user could have a dedicated agent :

`SuperMASProjectAgent`

This agent could have access to :

— the project files ;

— a description of the project's objectives ;

— the list of members ;

— information relating to tasks or deadlines ;

— previous email exchanges for the project ;

— the discussion thread currently being processed.

Its *system prompt* could also contain specific instructions provided by the user, for example :

— respond concisely ;

— systematically remind recipients of certain deadlines ;

— never commit to a new task without validation ;

— adopt an informal tone with certain partners ;

— prefer English when the project involves international partners.

If an email relating to `SuperMASProject` is received, the process could be as follows :

1. receipt of the email ;
2. analysis of the message by the routing agent ;
3. identification of the `SuperMASProject` context ;
4. selection of `SuperMASProjectAgent` ;
5. retrieval of the relevant context ;
6. retrieval of previous exchanges in the discussion thread ;
7. generation of a proposed response by the local LLM ;
8. transmission of the response to the coordinating agent ;
9. presentation of the proposal to the user ;
10. the user only has to validate or reject the proposal. If the user `validates` it, the email is sent ; otherwise, the agent must generate a new proposal while taking into account any comments the user may provide.

7. Minimum functional architecture

At minimum, the application must support the following scenario :

Email received → Routing agent → Specialized agent → Local LLM → Routing agent → User

The exact architecture is deliberately left open.
You may in particular consider whether it would be appropriate to add :

— a validation agent ;

— an agent responsible for searching for relevant information ;

— an agent responsible for memory ;

— an agent responsible for reformulating or checking the response ;

— an agent supervising the organization ;

— several routing levels.

These extensions are not mandatory. However, they must be justified if they are introduced.

8. Use of a local LLM

An important project constraint is that text generation must rely on an **LLM capable of running locally**.
The choice of :

— the model ;

— the inference engine ;

— the communication protocol ;

— the prompting strategy ;

— the context-retrieval mechanism ;

is left to the students.
The Java application may communicate with the local inference engine through a local API or any other mechanism you consider appropriate.
Your architecture must nevertheless avoid a strong dependency on any particular model.
For example, it should be possible to replace one local model with another without having to rewrite the entire framework.

9. Conversation management

An important aspect of the project concerns the management of discussion threads.
When an agent replies to an email, it should not necessarily consider only the most recently received message.
Your framework must provide a mechanism for taking into account all or part of :

— previous messages ;

— responses already generated ;

— responses actually sent by the user ;

— the context associated with the conversation.

You must consider how this memory should be represented and how much information it is relevant to send to the LLM.

10. User interface

A graphical interface is **strongly encouraged**.
It may be implemented using the Java technology of your choice.
Ideally, the user should be able to :

— view existing agents ;

— create a new specialized agent ;

— modify an agent ;

— define its *system prompt* ;

— associate files or other context elements with it ;

— view a received email ;

— view the agent selected by the router ;

— view the proposed response ;

— accept, modify, or reject this response.

Additional features may naturally be proposed.

11. Important : assistance rather than blind sending

In a first version, it is preferable for the system to **propose a response** rather than send it automatically.
The user must retain final control over the message that is sent.
You may nevertheless discuss in your report the mechanisms that would be necessary to enable more advanced automation while limiting the associated risks.

12. Implementation constraints

The project must be developed primarily in **Java**.
Particular attention will be paid to the quality of the software architecture.
Your framework must comply with the fundamental principles of object-oriented programming, including :

— encapsulation ;

— abstraction ;

— low coupling ;

— high cohesion ;

— separation of responsibilities ;

— extensibility ;

— reusability.

Students must also consider the appropriate use of **Design Patterns**.
Depending on the architectural choices, some patterns may for example be relevant :

— Strategy ;

— Factory / Abstract Factory ;

— Observer ;

— Mediator ;

— Command ;

— Chain of Responsibility ;

— Template Method ;

— Adapter ;

— Facade ;

— Repository.

The objective is obviously **not to artificially use as many Design Patterns as possible**. The patterns used must address a genuine need, and their use must be explained in the report.

13. Framework design

The objective is not only to produce a functional demonstration.

Your work must lead to the design of a **sufficiently generic multi-agent framework** that easily allows :

— the addition of new specialized agents ;

— the modification of their prompts ;

— the modification of their context sources ;

— the replacement of the LLM used ;

— the addition of new routing strategies ;

— the addition of new memory mechanisms ;

— the evolution of the multi-agent organization.

The extensibility of your solution will therefore be an important part of the assessment.

14. Design questions to be studied

Your report must in particular show that you have considered the following questions :

1. What is an agent in your architecture ?
2. What are its responsibilities ?
3. What information does it possess ?
4. How do the agents communicate ?
5. How does the routing agent determine the appropriate agent ?
6. What happens when several agents appear to be suitable ?
7. What happens when no agent is sufficiently suitable ?
8. How are the different contexts represented ?
9. How is the history of a conversation managed ?
10. How do you avoid unnecessarily sending too much context to the LLM ?
11. How can a new agent be added without substantially modifying the application ?
12. How can the LLM be changed ?
13. Which data must be persistent ?
14. What are the advantages and limitations of your multi-agent approach ?

15. Experiments

Your system must be tested on several scenarios.

You must in particular demonstrate that your system is able to distinguish between several types of emails.

For example :

— a student's question about a course ;

— a request from a fellow lecturer ;

— an administrative request ;

— an exchange related to the `SuperMASProject` project ;

— an ambiguous message that may correspond to several categories.

It will be useful to analyze :

— routing quality ;

— the relevance of the responses ;

— the influence of the provided context ;

— the influence of the *system prompt* ;

— observed errors ;

— the limitations of the chosen local LLM.

Students are encouraged to build a small test set allowing them to compare several configurations of their system.

16. Deliverables

Each group must submit the following items.

16.1. Source code

The complete Java project must be provided.
The repository must make it possible to clearly understand :

— the organization of the code ;

— the dependencies ;

— how to compile the project ;

— how to run the project ;

— how to configure the local LLM.

A `README` file is therefore expected.

16.2. Report

The report must present at least :

1. the problem studied ;
2. the objectives of the system ;
3. the selected multi-agent organization ;
4. the overall architecture of the application ;
5. the main design choices ;
6. the important classes and interfaces ;
7. the communication mechanisms between agents ;
8. the routing strategy ;
9. context management ;
10. discussion-thread management ;

11. integration of the local LLM ;
12. the Design Patterns used and their justification ;
13. the experiments carried out ;
14. the results obtained ;
15. the limitations of the solution ;
16. possible improvements.

Diagrams are strongly recommended, in particular :

— UML class diagrams ;

— sequence diagrams ;

— diagrams presenting the organization of the agents.

17. Group work and individual contributions

The project may be completed **in a group**.
However, the participation of each group member must be identifiable.
The report must therefore include a section entitled :

Contributions of group members

For each student, this section must specify :

— first name and last name ;

— student number ;

— the parts of the project to which the student contributed ;

— the main developments or tasks carried out.

A table may, for example, be used :

Student	**Student number**	**Contribution**
First name Last name	12345678	Agent architecture, router development, testing
First name Last name	87654321	Graphical interface, context management, LLM integration

This description must reflect the work actually carried out.

18. Important instruction concerning submission

WARNING

The report must contain the complete list of group members as well as the **student number of each member**.

ONLY ONE student per group must submit the project.

The other group members must not submit a second copy of the same project.

19. Assessment criteria

The assessment will in particular be based on :

— the relevance of the multi-agent modelling ;

— the quality of the proposed organization ;

— the relevance of the routing strategy ;

— the quality of agent specialization ;

— context and memory management ;

— integration of the local LLM ;

— the quality of the Java architecture ;

— compliance with object-oriented programming principles ;

— the relevance of the Design Patterns used ;

— the extensibility of the framework ;

— the quality of the proposed interface ;

— the quality of the experiments ;

— the quality of the report ;

— the ability to justify technical and scientific choices.

A more complex project is not automatically a better project.

A relatively simple solution with a clear, coherent, extensible, and properly justified architecture is preferable to an unnecessarily complex architecture.

20. Possible extensions

Groups wishing to go further may study, among other things :

— the use of RAG mechanisms (*Retrieval-Augmented Generation*) ;

— the use of local embeddings ;

— short-term and long-term memory ;

— learning the user's preferences ;

— dynamic addition of agents ;

— collaboration between several agents on the same response ;

— a critic agent responsible for evaluating a response before proposing it ;

— an estimate of the router's confidence level ;

— logging agent decisions ;

— routing explainability ;

— a comparison between a single-agent architecture and a multi-agent architecture ;

— mechanisms strengthening confidentiality and isolation between the different contexts.

These features are possible extensions and do not replace the main functionalities required by the project specification.

21. Conclusion

This project should be approached as a **multi-agent system design** project before being considered as a simple LLM-use project.

The use of a language model is one component of the system.

The main challenge is to consider how several specialized agents can organize themselves, communicate, share certain information, and use different knowledge sources in order to collectively solve a complex problem.

Your work must therefore show how the concepts studied in the Multi-Agent Systems course make it possible to propose an architecture that differs from a simple application sending every email to a single language model.

Good luck with the project !